EECS 111 System Software Spring 2026
Project 1: Fork & Pthread

Due Date (April 21st, 2026 11:59 PM)

Project Overview

In this project, you will demonstrate how to manipulate multiple processes and threads to process input data in parallel. The goal is to build a program that reads multiple CSV files containing student IDs and grades, sorts the data, and computes summary statistics, all while **leveraging the parallelism concepts covered in lecture.**

Your program receives two command-line arguments: the number of child processes to create and the number of threads ***each*** child process should spawn. The main process divides the input files among its children, each child sorts its assigned file(s) using a multithreaded merge sort, and then computes three statistics (Average, Median, Standard Deviation) for the grades in each file. The final output consists of 2 CSV files for each input file: a ranked list of students sorted by grade (highest first) and a statistics summary.

This project directly exercises the process and thread hierarchy discussed in lecture. You will use fork to create child processes, mirroring the parent-child relationship and process lifecycle (creation, execution, termination) from the lecture's coverage of operations on processes. Within each child, you will use the POSIX Pthreads library to partition data across threads for parallel sorting. You will also see firsthand the distinction between what threads share and what they own privately, and why this matters when dividing work across threads without locks.

The project contains a coding task, as well as **conceptual questions about your work**. These questions are designed to test your understanding of your own implementation. It is worth taking a look at them before you start writing your code, as they might offer some insight into the underlying topics.

Project Specification

There are several requirements you must satisfy to receive a perfect score. Please read this description carefully and understand the full specification before you begin programming. You will see the following files in the ***p1*** directory:

1. `./input/`: This directory includes CSV files for input data.
2. `./output/`: This directory is for output data.
3. `main.cpp`: This is a top file for this project.
4. `p1_process.h`: This is a header file to handle processes.
5. `p1_process.cpp`: This is a file to handle processes.
6. `p1_threads.h`: This is a header file to handle threads.
7. `p1_threads.cpp`: This is a file to handle threads.
8. `Makefile`: This is the compilation makefile.
9. `autograder.py`: This is a Python program that allows you to do self-evaluation.
10. `questions.txt`: This is a file containing the conceptual questions, and space for you to write your answers

Here is the description of how your program (`p1_exec`) should behave:

- `p1_exec` receives two arguments: 1) the specified number of child processes and 2) the number of threads per child process. E.g: `` `./p1_exec 3 4` `` should spawn 3 processes, each with 4 threads.
- Your program needs to create multiple processes to handle multiple files in parallel. Each child process handles at least one file. In other words, one single child process may handle multiple files.
- For each input file, you will need to first sort the input data read from a file with a multithreaded sorting algorithm (merge sort is suggested, see https://en.wikipedia.org/wiki/Merge_sort)
- Then, calculate the following statistics: Average, Median, and Standard Deviation.
- Only the child process should create multiple threads to perform the sorting. So, the process hierarchy will be the following: Main process (spawns) -> Child processes (spawns) -> Threads

You must not waste any resources; thus, you should not create any non-working processes or threads.

- The results need to be saved to the output directory with the following filename: `<input filename>_stats.csv` and `<input filename>_sorted.csv` (i.e., os_stats.csv and os_sorted.csv).
  - NOTE: The separator here is an underscore (_), not a hyphen
- When a process is created and terminated, you should print a message (see below) with its process id to indicate the process has been created/terminated (you don't need to do this for threads).
- The main process must complete its behavior after its child processes.

The complete program will provide the following output. Pay attention to the order in which processes terminate. (The main process terminates last)

```
./p1_exec 2 4
Main process is created. (pid: xxxx)
Child process is created. (pid:
yyyy) Child process is created.
(pid: zzzz)
...
Child process is terminated. (pid:
yyyy) Child process is terminated.
(pid: zzzz) Main process is terminated.
(pid: xxxx)
```

The complete program will provide the following output files:

os-stats.csv content: (please display at least 3 decimal places)

```
Average,Median,Std. Dev
XX.XXX,XX.XXX,XX.XXX
```

os-sorted.csv content: (Notice that rank starts at 1, and don't worry about the formatting for the # of digits to display after the decimal)

```
Rank,Student ID,Grade
 1,XXXXXXXXXX,XX.XXX
 2,XXXXXXXXXX,XX.XXX
 3,XXXXXXXXXX,XX.XXX
 …
```

During this project, you may change the given source code (p1_process.cpp and p1_thread.cpp) or add any new header and source files. However, please remember that you must apply corresponding changes to the `Makefile` and ensure your program remains compilable with the `'make'` command. Compiling your code using the `'make'` should yield the `p1_exec` file as the executable. To run the self-evaluation, you can run the `'make test`, but it will give you some exceptions when you have not started the implementation. You can use `'make clean'` to clean files generated during the compilation, which can be needed when submitting your finished project. Nevertheless, when compiling successfully, you will get an executable (`p1_exec`) for this project.

Notes

- Please refer to the lecture slides or this page for more pthread usage information:
  - https://www.cs.cmu.edu/afs/cs/academic/class/15492-f07/www/pthreads.html
- In the context of this project, multithreaded sorting can be implemented by splitting the input file and assigning each thread a slice of the file
  - Make sure that when splitting the input file, every line in the file is sorted
- It may be helpful to create a class to use for managing threads and sorting, however, pthread_create will not be able to take class member functions. See the following page for more information
  - https://stackoverflow.com/questions/1151582/pthread-function-from-a-class
- Even though you can generate a correct output, that does not mean that your code considers some extreme cases. You should verify your code with some corner cases.

- There will always be exactly five inputs with the names currently given to you in the project 1 package.
- If two students have the same grade, the order does not matter. As long as you get a small number of differences for the benchmarks it means that your code is working. The benchmarks that are going to be used for grading will not have reparation in them.
- The size of the input files is not constant. You should not make any assumptions about the length of the input files.
- The time out for sorting an input with 1 million entries is set to be 10 minutes. Make sure that your program can carry the task under this limit.

If you have any questions regarding the project, please ask them during the discussion session and/or on Ed Discussion Board.

Guidance

This project can be broken down into multiple stages. It is recommended to work on each stage one after the other so you have a base to work off of

1. (p1_process.cpp) The first stage of the project consists of I/O or input and output. We expect your implementation to read from a file and output to a new file, therefore the first step you can take is to read the file into a data structure. Since we're using C++, we have access to built-in data structures (<vector>) along with abstraction via the use of structs and classes. It may be helpful to transform the input data of a student id and a grade into a struct/class, that way the student ID and grade are grouped together. (Note that this is provided in the starter code)
    a. At the end of this stage, you should be able to output the input CSV file back to the console.
2. (p1_process.cpp) Once this is done, you can work on outputting the data structure to the output file. In C++, we have two options for file I/O. We can use C style file I/O (<cstdio>) using fopen, fprintf, fscanf, fgets, fputs (see this or this for more reference). Another method is to use C++ style file I/O (<fstream>) using ifstream and ofstream (see this for reference).
    a. At the end of this stage, you should be able to output the input CSV file to another CSV file (in the output directory), where the contents are the same.
3. (p1_process.cpp, p1_threads.cpp) The next stage consists of sorting the data and generating statistics. For this, it is recommended to implement merge sort. You want to sort the students in order of their grade descending (highest first). The implementation of merge sort won't be mentioned as there exist many resources on this topic already (see the link above).
    a. At the end of this stage, you should be able to output the sorted input CSV file to another CSV file (in the output directory). You may also run the autograder to check that this is true.
4. (p1_process.cpp) After this is accomplished, you can start working on the multiprocess aspect of the project. The project specifies that we should create a number of child processes where each child process is assigned a subset of the input files (to create a child process, we must first fork, then we can have the child process call a function that performs the sorting). This subset can be larger than one file. To tackle this problem, you must first decide how to split the input files into these subsets. The easiest way to accomplish this is to divide the number of input files by the number of processes. This may result in one process having more files than the rest due to integer division (make sure all files are processed).

    a. At the end of this stage, this stage should mimic the same functionality as the stage before.
5. (p1_threads.cpp) Finally, the last stage is implementing multithreading. Each child process will spawn multiple threads (the number of threads is as defined in the arguments of the executable). To perform multithreaded sorting, you want to split the original list n ways, where n is the number of threads. Therefore, each thread will be assigned a subset of the original list with size = (size of list / number of threads) to sort. Each thread will then run merge sort. Once all the threads are finished, you will be left with (number of threads) sublists that are all sorted. You now need to merge the resulting sorted sublists together. To tackle this, remember that merge sort's merge function takes two sorted lists and merges them together to return one sorted list. You need to keep merging the resulting sublists until there are no more sublists, and it is easiest to implement this iteratively.
    a. Hint: In each iteration of the merge, you reduce the number of unmerged sublists by 1 since one of the sorted sublists is now a larger sorted sublist.
    b. At the end of this stage, your project should be complete. Make sure to run the autograder to verify the output format, but write your own tests to validate against edge cases.

Conceptual Questions

Question 1 Shared vs Private variables:

Identify one variable in your p1_threads.cpp that is shared across threads and one that is private to each thread. For the shared variable, explain why concurrent access does not cause a race condition in your implementation.

Question 2 Resource waste:

If the user runs `` `./p1_exec 7 3` `` with 5 input files, how many child processes should your program actually create? Point to the line(s) in your code where this adjustment is handled. What could go wrong if all 7 processes were created without accounting for the number of files?

Question 3 Data Parallelism and fork:

Look at the section of your code where you call fork(). Are you reading the input CSV files before or after the fork? What is the downside of loading all input files before forking? *Hint: think about which files each child process actually needs, and what fork() copies from the parent's address space.*

Submission Guidelines

You need to compress all the files in the ***p1*** folder into a single archive **.zip file** and upload it to Gradescope. You can update your submission till the deadline and your last submission will be graded. Since your submissions will be processed by a program, there are some very important things you must do, as well as things you must not do:

1. Write your answers to the conceptual questions Q1-Q3 in the plain text file **questions.txt** and include it in your submission.
2. When zipping your source code, use the following command in a Linux/Unix terminal while you are inside the directory with your source code **and the file containing the answers to the conceptual questions**:
   a. zip -j p1.zip <list of your files>
   b. Example for above:
   `zip -j p1.zip questions.txt main.cpp p1_process.h p1_process.c …`
3. It is imperative to keep the output format **the same** as what you are asked for.You MUST USE only the **C++98** standard. If you use any new features like C++11 or C++14, your code will **not be graded**.
4. After submitting, download your submission to verify if all required files are uploaded. Make sure your code can be compiled and run. If we cannot compile your code, then your code will not be tested and graded.
5. Your executable filename must be ***`p1_exec`***.
6. Your submission must contain all your source/header files/Makefile in the root of the zip file. It is suggested to remove all the compiled object files and the executable file before submitting with the following command: `make clean`.
7. Since the input and output file sizes are large, you **MUST NOT** include input or output files in your submission.
8. A sample auto-grader script which is written in python is provided for you. You can use it to double-check your output format. However, the actual auto-grader script is different. It will use other test benches, will run in a sandbox, and will have a software similarity detection feature. Similarity detection features will compare your source codes with the other source codes available online. If they are similar enough, it will give us a warning about it. So, you can talk to each other about the project, and visit online resources, but you must write your own code.

Grading

1. (5%) Following the submission format
2. (10%) Compile your code with your Makefile without any problem.
3. (10%) Command-line output matches the description for any type of input.
4. (45%) Output files match the description for any input. **(Passes the auto-grader test)**
   - (-25%) If there is no multithreading
   - (-25%) If there is no multiprocessing
5. (30%) Answers to the conceptual questions.